데이터베이스(SQL)

10장. Window Function

학습 목표

1) Window Function과 GROUP BY의 차이를 설명할 수 있다.
2) OVER 절의 PARTITION BY, ORDER BY, 프레임의 역할을 구분할 수 있다.
3) 순위 함수(ROW_NUMBER, RANK, DENSE_RANK)의 동점 처리 차이를 설명할 수 있다.
4) LAG, LEAD가 각각 어떤 질문에 쓰이는지 설명할 수 있다.
5) ROWS와 RANGE 프레임의 차이와 기본 프레임의 동작을 설명할 수 있다.

전체 흐름

1) Window Function이란 → 2) OVER 절 → 3) 함수의 종류 → 4) Window Frame

01 Window Function이란

윈도우 함수는 왜 필요할까?

1) 각 관측값의 토크는 같은 제품 품질 유형의 평균보다 높은가?

2) 카테고리마다 가장 비싼 제품 2개는?

3) 1월부터 이번 달까지의 누적 금액은?

4) 1번 설비의 진동값은 직전 관측보다 얼마나 달라졌는가?

Window란

현재 행과 관련된 여러 행을 계산에 함께 참조하기 위한 논리적 범위다.

예시 데이터 (가상 예)

제품	유형	가격
A	음료	10
B	음료	20
C	음료	30
D	조미료	40

현재 행: B (음료, 20)

참조하는 관련 행(현재 행 포함)

음료 10 · 20 · 30

이 Window로 평균을 구하면

(10 + 20 + 30) ÷ 3 = 20

D(조미료)의 Window는 조미료 40 한 행뿐이다.

개별 행을 유지한 계산

앞의 Window(같은 유형의 행)마다 평균을 구해 각 행 옆에 붙인다.

원본

제품	유형	가격
A	음료	10
B	음료	20
C	음료	30
D	조미료	40



Window Function 적용 후

제품	유형	가격	유형 평균
A	음료	10	20
B	음료	20	20
C	음료	30	20
D	조미료	40	40

각 행에 자기 유형의 평균이 열 하나로 붙는다. 그래서 “내 가격이 유형 평균보다 높은가”를 행마다 바로 비교할 수 있다.

GROUP BY와 Window Function의 차이

GROUP BY

같은 값끼리 그룹으로 요약한다.

- 각 그룹당 결과 한 행
- 그룹별 합계·평균 보고서

4개 원본 행 → 2개 그룹

GROUP BY 유형

유형	유형 평균
음료	20
조미료	40

Window Function

각 행을 그대로 두고 Window 안의 행으로 계산한다.

- 각 조회 행에 계산 결과 열 추가
- 비교·순위·누적·이전 값 계산에 사용

4개 조회 행 → 4개 행 + 계산 열

AVG(가격) OVER (PARTITION BY 유형)

제품	유형	가격	유형 평균
A	음료	10	20
B	음료	20	20
C	음료	30	20
D	조미료	40	40

Window Function의 정의

Window Function (윈도우 함수)

현재 행과 관련된 행 집합을 대상으로 계산한 값을, 조회 결과의 각 행에 돌려주는 함수다.

- 일반 집계 함수 AVG, SUM, COUNT도 OVER와 결합하면 Window 집계로 쓸 수 있다.
- ROW_NUMBER, RANK, LAG처럼 Window 전용 함수도 있다.
- “Window”는 계산의 범위이고, “함수”는 평균·순위·이전 값처럼 실제로 계산하는 기능이다.

용어정리

윈도우(현재 행과 관련된 행의 범위)는 OVER( ) 안의 파티션 + 정렬 + 프레임으로 정한다.

현재 행 (Current Row)

지금 계산 결과를 붙이고 있는 행

예) B (음료, 20)

파티션 (Partition)

PARTITION BY로 나눈 같은 그룹

예) B와 같은 음료 → A · B · C

정렬 (Window ORDER BY)

파티션 안에서 행을 줄 세우는 순서

예) 가격 순 → A(10) → B(20) → C(30)

프레임 (Window Frame)

파티션 안에서 실제로 계산에 쓰는 행

예) 가격 순일 때 B의 프레임 → A · B

02 OVER 절

OVER 절의 기본 구조

함수명(인수) OVER ([PARTITION BY 열] [ORDER BY 열] [프레임 지정])

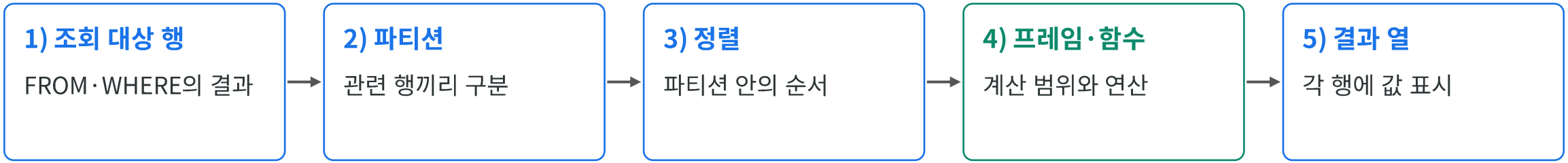
PARTITION BY
관련 행을 어떤 그룹으로 나눌지 정한다.
생략하면 전체 행이 하나의 파티션이다.

ORDER BY
파티션 안의 계산 순서를 정한다.
화면 표시 순서와는 별개다.

프레임 지정
누적·이동 계산의 행 범위를 정한다.
ROWS, RANGE, GROUPS를 쓴다.

용어	뜻
OVER	Window의 범위와 순서를 정하는 절
PARTITION BY	Window를 그룹으로 나눔. 행 수는 줄지 않음
ORDER BY	그룹 안에서 계산 순서를 정함
ROWS BETWEEN	계산에 넣을 행의 범위를 지정함
Frame	현재 행 기준 계산에 넣는 행의 범위

Window 계산을 이해하는 논리적 흐름



OVER()가 비어 있으면

파티션과 정렬을 지정하지 않은 Window 집계다. 전체 행이 하나의 Window가 된다.

직원	급여	전체 평균
A	100	200
B	200	200
C	300	200

```
AVG(salary) OVER ()
```

세 행에 같은 전체 평균 200이 붙는다.

PARTITION BY — 그룹별 계산

직원	부서	급여	부서 평균
A	영업	100	200
B	영업	200	200
C	영업	300	200
D	개발	150	250
E	개발	350	250

```
SELECT
  depname,
  empno,
  salary,
  avg(salary) OVER (PARTITION BY depname)
FROM empsalary;
```

영업부 평균: (100 + 200 + 300) / 3 = 200

개발부 평균: (150 + 350) / 2 = 250

같은 부서의 행만 써서 평균을 낸다.

ORDER BY 의미 차이

OVER 안에서는 계산 순서, SELECT 문 끝에서는 조회결과 순서다.

OVER 안의 ORDER BY

LAG가 참조하는 이전 행, ROW_NUMBER의 번호 순서, 누적 계산의 순서를 정한다.

```
SUM(amount) OVER (ORDER BY month)
```

SELECT 끝의 ORDER BY

계산이 끝난 결과 행을 화면에 어떤 순서로 보일지 정한다.

```
ORDER BY customer_name
```

03 함수의 종류

Window Function 종류

집계형

SUM · AVG · COUNT

관련 행들의 합계·평균·건수

이전·다음 값

LAG · LEAD

정렬 순서상 앞뒤 행의 값 참조

순위형

ROW_NUMBER · RANK · DENSE_RANK

그룹 안에서 순서·순위를 부여

ROW_NUMBER · RANK · DENSE_RANK

이름	점수	ROW_NUMBER	RANK	DENSE_RANK
A	100	1	1	1
B	90	2*	2	2
C	90	3*	2	2
D	80	4	4	3

ROW_NUMBER: 연속 번호 · RANK: 동점 뒤 순위를 건너뜀 · DENSE_RANK: 동점 뒤 순위를 건너뛰지 않음

동점인 행과 보조 정렬 기준

OVER의 ORDER BY 값이 같은 행을 같은 값(동점)인 행(= peer)으로 본다.

상품	점수	그룹
A	100	첫 번째 값
B	90	동점 그룹
C	90	동점 그룹
D	80	마지막 값

보조 정렬 기준 (tie-breaker) 추가

ROW_NUMBER처럼 동점 안의 순서가 중요하면 고유 ID 등을 ORDER BY에 더해 순서를 확정한다.

그룹별 상위 N개를 구하는 논리

순위를 계산한 뒤 그 결과를 다시 걸러낸다.

1) 그룹을 구분
부서별로 행을 나눈다.

```
PARTITION BY dept
```

→

2) 그룹 안에서 순번 부여
점수가 높은 순으로 1, 2, 3 …

```
ROW_NUMBER() OVER (
  PARTITION BY dept
  ORDER BY score DESC)
```

→

3) 상위 N개만 남김
순번이 3 이하인 행만 고른다.

```
WHERE rn <= 3
(바깥 SELECT에서)
```

순번(rn)은 같은 SELECT의 WHERE에서 바로 걸러낼 수 없다.
그래서 CTE나 인라인 뷰(FROM 절 서브쿼리)로 계산 단계와 선택 단계를 나눈다.

Window Function은 논리적으로 WHERE·GROUP BY·HAVING 이후에 계산된다.

FROM → WHERE → GROUP BY → HAVING → **SELECT (Window Function 계산)** → ORDER BY

CTE와 Window Function을 함께 사용하는 예

내부 단계 — 계산

```
WITH ranked AS (
  SELECT id, dept, score,
         ROW_NUMBER() OVER (
           PARTITION BY dept       -- 1) 그룹 구분
           ORDER BY score DESC, id -- 2) 순번 기준
         ) AS rn
  FROM example_scores
)
```

외부 단계 — 선택

```
SELECT dept, score, rn
FROM ranked
WHERE rn <= 3;        -- 3) 상위 3개만
```

인라인 뷰로 쓰면: … FROM (내부 SELECT) AS ranked WHERE rn <= 3;

ranked 결과

id	dept	score	rn	결과
1	영업	95	1	**남김**
2	영업	90	2	**남김**
3	영업	90	3	**남김**
4	영업	80	4	제외
5	개발	92	1	**남김**
6	개발	88	2	**남김**

rn은 부서마다 1부터 다시 시작
90 으로 점수가 같은 경우 보조 정렬 기준 id로 정렬

LAG와 LEAD — 이전·다음 행 참조

정렬 기준으로 현재 행 앞뒤에 있는 다른 행의 값을 가져온다.

관측 시점	현재 값	LAG 이전 값	LEAD 다음 값
09:00	10	NULL	14
10:00	14	10	12
11:00	12	14	16
12:00	16	12	NULL

```
LAG(value) OVER (ORDER BY observed_at)
LEAD(value) OVER (ORDER BY observed_at)
```

첫 행에는 이전 행이 없고, 마지막 행에는 다음 행이 없다. 그 자리는 NULL이 된다.

LAG·LEAD를 해석할 때 필요한 조건

데이터의 순서가 실제 분석 질문과 연결되어야 한다.

시간을 기준으로 정렬

실제 관측 시각(datetime 등)을 OVER의 ORDER BY에 지정한다.

같은 시각의 중복 처리

같은 시각에 여러 행이 있으면 ORDER BY에 보조 정렬 기준을 추가해 순서를 확정한다.

분석 단위 구분

설비별 이전 값을 원하면 PARTITION BY machineid를 넣는다.

불규칙한 관측 간격

이전 행이 "정확히 1시간 전"을 뜻하지는 않는다. 정기 측정인지 사건 기록인지 간격을 먼저 확인한다.

04 Window Frame

Window Function 종류별 참조 범위

함수 종류	주로 사용하는 기준	결과
SUM · AVG (윈도우 집계)	현재 프레임	누적·이동 계산
ROW_NUMBER · RANK	파티션 + 정렬	순번과 동점 순위
LAG · LEAD	파티션 + 정렬	이전·다음 행

Window Frame의 경계 문법

현재 행을 기준으로 앞뒤 몇 행을 계산에 넣을지 지정한다.

이전 2행
2 PRECEDING

이전 1행
1 PRECEDING

현재 행
CURRENT ROW

다음 1행
1 FOLLOWING

다음 2행
2 FOLLOWING

```
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW   -- 초록 두 칸 + 주황 한 칸 = 3행
```

경계	ROWS (행 개수로 센다)	RANGE (정렬값으로 센다)
UNBOUNDED PRECEDING	파티션의 첫 행부터	같다
n PRECEDING	현재 행의 앞 n행	**정렬값이 n만큼 작은 행부터**
CURRENT ROW	현재 행	**현재 행 + 동점 행**
n FOLLOWING	현재 행의 뒤 n행	**정렬값이 n만큼 큰 행까지**
UNBOUNDED FOLLOWING	파티션의 마지막 행까지	같다
BETWEEN 시작 AND 끝	시작 경계부터 끝 경계까지	같다

누적합(Running Total)의 원리

정해진 순서에 따라 첫 행부터 현재 행까지 더한다.

월	월별 매출	누적합	계산
1월	10	10	10
2월	20	30	10 + 20
3월	30	60	10 + 20 + 30
4월	40	100	10 + 20 + 30 + 40

```
SUM(amount) OVER (
  ORDER BY month
  ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
)
```

누적의 의미가 생기려면 ORDER BY에 시간이나 순번처럼 순서가 있는 값을 지정한다.

공식 문서: 윈도우 함수 (튜토리얼 3.5)

이동평균(Moving Average)

현재 행이 한 칸씩 내려가면서 일정한 개수의 최근 행을 평균 낸다.

월	매출	최근 3행의 평균	포함되는 월
1월	10	10	1월
2월	20	15	1·2월
3월	30	20	1·2·3월
4월	40	30	2·3·4월

```
AVG(amount) OVER (
  ORDER BY month
  ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
)
```

누적합과 같은 틀에서 시작 경계만 2 PRECEDING으로 바꾼다.

앞쪽 행은 행이 부족하므로 있는 행만으로 평균을 낸다. 1월은 1월만, 2월은 1·2월만이다.

기본 프레임과 동점 행

OVER에 ORDER BY만 쓰고 프레임을 생략하면, **정렬값이 같은 행까지** 함께 포함된다.

행	정렬값	금액	기본 프레임 누적합
A	1	10	10
B	2	20	50
C	2	20	50
D	3	30	80

왜 B·C 모두 50인가?

B와 C는 정렬값이 2로 같은 동점(peer)이다.
기본 프레임은 동점 행을 함께 넣으므로, B를 계산할 때 C까지 더한다.

B: 10 + 20 + 20 = 50
C: 10 + 20 + 20 = 50
D: 10 + 20 + 20 + 30 = 80

```
SUM(amount) OVER (ORDER BY sort_key)   -- 프레임 생략
```

ROWS와 RANGE의 차이

같은 데이터라도 프레임 방식에 따라 누적합이 달라진다.

행	정렬값	금액	1) ROWS	2) RANGE	3) 프레임 생략
A	1	10	10	10	10
B	2	20	**30**	**50**	**50**
C	2	20	50	50	50
D	3	30	80	80	80

ROWS

행의 위치로 범위를 정한다.
현재 행까지 한 행씩 센다.

RANGE

정렬값의 관계로 범위를 정한다.
정렬값이 같은 행(동점)을 함께 넣는다.

쓰는 법	OVER ( ) 안에 쓰는 내용
1) ROWS	`ORDER BY sort_key, id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`
2) RANGE	`ORDER BY sort_key RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`
3) 프레임 생략	`ORDER BY sort_key` **→ 프레임을 생략하면 RANGE로 동작한다. 결과가 2)와 같다.**

ROWS는 A·B 두 행만 더해 30, RANGE는 동점인 C까지 더해 50이다.

행 수와 기간의 차이

관측 날짜	측정값	1) ROWS 최근 3행 합	2) RANGE 최근 3일 합
10월 1일	10	10	10
10월 2일	20	30	30
10월 5일	30	60	30

빠진 날짜의 영향

10월 3일과 4일은 관측값이 없다.

ROWS 최근 3행은 1·2·5일을 더해 60이고, RANGE 최근 3일은 3~5일만 봐서 30이다.

쓰는 법	OVER ( ) 안에 쓰는 내용
1) ROWS 최근 3행	`ORDER BY obs_date ROWS BETWEEN 2 PRECEDING AND CURRENT ROW`
2) RANGE 최근 3일	`ORDER BY obs_date RANGE BETWEEN INTERVAL '2 days' PRECEDING AND CURRENT ROW`

“최근 N개 관측”은 ROWS, “최근 N일”은 RANGE + INTERVAL로 구한다.

05 해석과 선택

분석 질문과 함수 선택

분석 목적에 맞춰 함수를 고른후 분석단위와 정렬기준을 선택한다.

질문	함수	확인할 조건
개별 제품 vs 품목 평균	AVG() OVER (PARTITION BY...)	제품군 정의
부서별 매출 순위	RANK / ROW_NUMBER	동점 처리
월별 누적 매출	SUM() OVER (ORDER BY...)	월 집계·순서
설비의 전회 센서값	LAG() OVER (...)	설비 ID·측정 시각

공식 문서: 윈도우 함수 (9.22)

GROUP BY · Window · CTE 선택 기준

세 개념은 대체 관계가 아니라 서로 보완하는 구성 요소다.

요약 결과가 필요하다

GROUP BY

그룹별 건수·합계·평균을 몇 행으로 요약

개별 행과 함께 계산한다

Window Function

행별 비교·순위·누적·이전 값

계산 결과를 다시 가공한다

CTE / 서브쿼리

단계를 나누고 결과를 다음 필터·집계에 활용

핵심 정리

1) Window Function은 관련 행을 참조해 계산한 값을 각 행 옆에 붙인다. 행 수는 줄지 않는다.

2) GROUP BY는 그룹마다 한 행으로 요약하고, Window Function은 개별 행을 유지한 채 계산한다.

3) OVER 안에서 PARTITION BY는 그룹 구분, ORDER BY는 계산 순서, 프레임은 집계형 함수의 계산 범위를 정한다.

4) OVER 안의 ORDER BY는 계산 순서이고, SELECT 끝의 ORDER BY는 결과를 보여 주는 순서다.

5) Window Function은 집계형(SUM·AVG·COUNT), 순위형(ROW_NUMBER·RANK·DENSE_RANK), 이전·다음 값(LAG·LEAD)으로 나눈다.

6) ROW_NUMBER는 동점에도 연속 번호, RANK는 동점 뒤 순위를 건너뛰고, DENSE_RANK는 건너뛰지 않는다.

7) 동점 안의 순서가 중요하면 ORDER BY에 고유 ID 같은 보조 정렬 기준을 추가해 순서를 확정한다.

8) Window Function은 WHERE에서 바로 쓸 수 없으므로, CTE나 인라인 뷰로 계산 단계와 선택 단계를 나눠 거른다.

9) LAG·LEAD는 시간 간격이 아니라 정렬 순서상 바로 앞뒤 행을 가져온다. 설비별로는 PARTITION BY로 나눈다.

10) 누적합은 ROWS BETWEEN을 적는다. 생략하면 같은 값(동점)인 행이 한꺼번에 더해질 수 있다.

공식 문서: 윈도우 함수 (튜토리얼 3.5) / 윈도우 함수 호출 (4.2.8) / 윈도우 함수 (9.22)